

Museum Heist: A Grid-Based Stealth Game Integrating Greedy Search, Backtracking, Linked Lists, and Binary Search Trees

Nelson de Jesus Navarro De La Rosa, Henry Samuel Garrido Medina, Josue Urrego Lopez
Computer Science I, Systems Engineering Program
Universidad Distrital Francisco Jose de Caldas, Bogota, Colombia

Abstract—Museum Heist is a Computer Science I project implemented as a playable grid-based stealth game. The project integrates Python, Pygame, C++, and JSON file communication to demonstrate course concepts through interactive mechanics rather than isolated examples. The player must collect valuable items, avoid cameras, guards, and vision zones, and reach the exit before losing by alarm, movement limit, or time limit. The C++ engine validates movement and owns the official game state, while Python renders the user interface, executes greedy item recommendation, and computes a backtracking escape route. The implementation also includes a custom linked list for movement history and a custom binary search tree for item values. The final game supports difficulty levels, vision zones, item collection, win/loss states, final metrics, and restart/main-menu controls.

Index Terms—Computer Science I, greedy algorithm, backtracking, linked list, binary search tree, Pygame, C++, JSON, grid game.

I. INTRODUCTION

Computer Science I projects should show that algorithms and data structures can solve real problems instead of remaining disconnected theory. Museum Heist was designed as a grid-based stealth game where the algorithms are direct game mechanics. The player moves through a museum, collects items, avoids surveillance, and escapes through an exit. This creates a constrained pathfinding problem with rewards and risk.

The implementation follows a multilayer architecture. C++ implements the rule engine and required data structures. Python implements user interaction, algorithm visualization, and the JSON bridge. Pygame renders the game screens. The project therefore connects algorithm design, recursion, memory-based data structures, file input/output, and graphical interaction in one system.

II. PROJECT REQUIREMENTS AND SCOPE

The implemented variant is Museum Heist. Its required mechanics are: a greedy item decision, a backtracking escape route, a movement log list, and an item value tree. Table I summarizes how the implementation satisfies these requirements.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the execution architecture. The game starts in `game/ui/main.py`. Pygame receives keyboard events and calls `bridge.py`. The bridge writes `input.json`, runs the

TABLE I: Requirement coverage

Requirement	Implementation
Greedy algorithm	<code>greedy.py</code> selects a remaining item using value and local danger criteria.
Backtracking algorithm	<code>backtracking.py</code> recursively searches a safe route to the exit.
Linked list	<code>MovementList</code> stores movement history in C++.
BST	<code>ItemBST</code> stores remaining items ordered by value.
JSON bridge	<code>input.json</code> and <code>state.json</code> connect Python and C++.
Pygame UI	<code>main.py</code> renders screens, board, HUD, route, and selected item.

C++ executable, and reads `state.json`. Python algorithms consume the loaded state for visualization, but the C++ engine remains responsible for official state transitions.

The execution cycle is as follows: (1) the UI captures a movement, reset, tick, or menu action; (2) Python writes the previous state and action to `input.json`; (3) C++ reads the action, validates it, updates state, and writes `state.json`; (4) Python reloads the state, recomputes greedy and backtracking outputs, and redraws the game.

IV. C++ ENGINE

The engine is implemented in `engine/main.cpp` and compiled with:

```
g++ -std=c++17 engine/main.cpp engine/linked_list.  
cpp engine/tree.cpp -o engine/museum_engine
```

The engine selects a hardcoded map according to difficulty. Table II summarizes the implemented difficulties.

TABLE II: Difficulty parameters

Difficulty	Grid	Alarm	Moves	Time
Easy	8x8	4	45	90 s
Normal	8x8	3	35	60 s
Hard	10x10	2	28	45 s

The engine validates each movement by checking boundaries, walls, camera cells, guard cells, and vision zones. Wall collisions keep the player in place and increase alarm. Entering a camera cell, guard cell, or vision zone causes game over. A

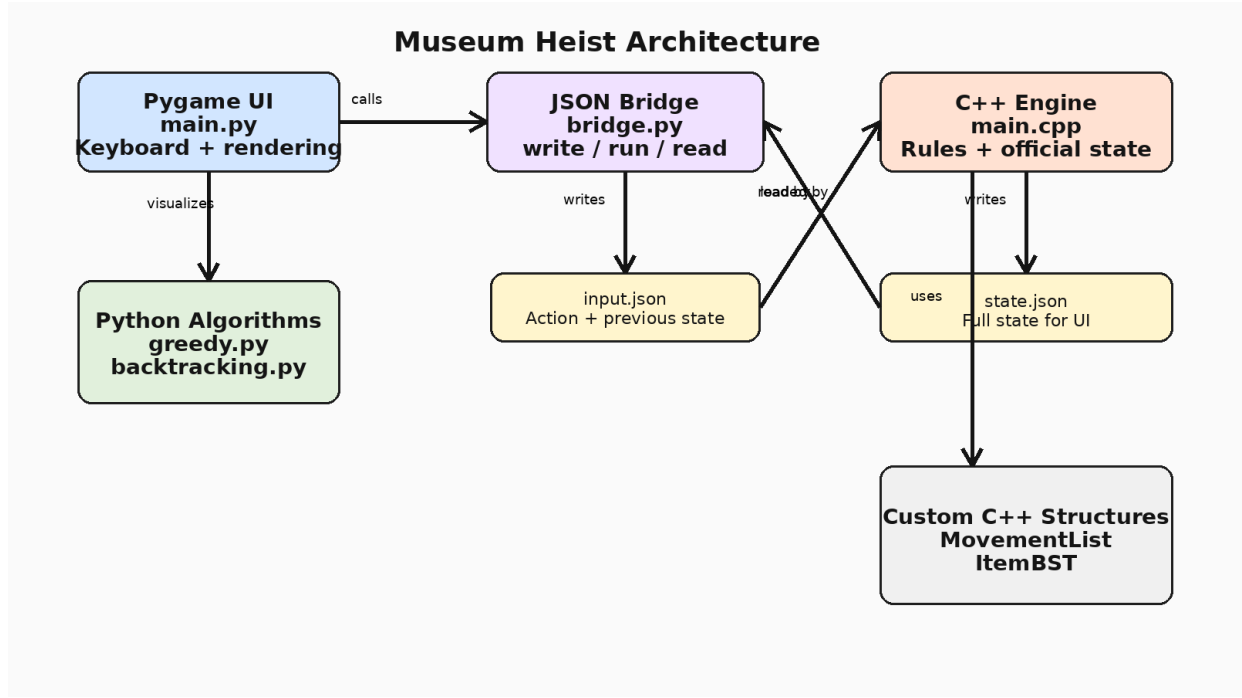


Fig. 1: File-based architecture connecting Pygame, Python algorithms, JSON bridge, C++ engine, and custom structures.

valid movement updates the player position and appends the new position to the movement history.

The engine also processes item collection. When the player reaches an uncollected item, its value is added to the score, its id is added to `collected_items`, and the item is omitted from the visible items array. The engine controls win/loss conditions: escape, caught in vision, alarm limit, movement limit, and time expiration.

V. JSON BRIDGE

The bridge is file-based. `input.json` stores the requested action and the previous state values that must be preserved, including difficulty, player position, movement history, collected items, score, escaped flag, game-over flag, alarm, and elapsed seconds. `state.json` stores the full state generated by C++, including grid, objects, vision zones, metrics, status, rank, movement history, and items sorted by value.

Listing 1: Simplified input example.

```

{
  "action": "move",
  "direction": "right",
  "difficulty": "Normal",
  "player": {"row": 0, "col": 0},
  "score": 0,
  "alarm": 0,
  "elapsed_seconds": 12
}

```

JSON was selected because it is readable, easy to inspect, and available in Python through the standard `json` module. The C++ engine manually reads the fields it needs and writes JSON with `std::ofstream`, keeping the implementation simple and explainable.

VI. DATA STRUCTURES

Fig. 2 summarizes the required C++ data structures. The project does not use `std::list`, `std::map`, or `std::set` for the required structures.

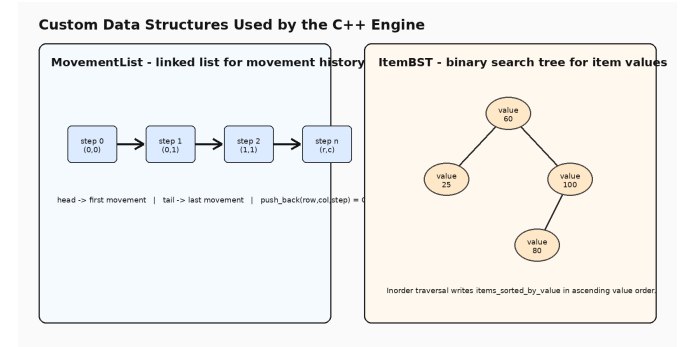


Fig. 2: MovementList and ItemBST as used in the C++ engine.

A. Movement List

`MovementList` stores a linked sequence of `MovementNode` elements. Each node stores row, column, step, and a pointer to the next node. The list maintains a `tail` pointer, so appending a valid move is $O(1)$. Traversing the list to write JSON is $O(n)$, and clearing the list is also $O(n)$.

B. Item Binary Search Tree

`ItemBST` stores remaining items ordered by value. Smaller values go to the left subtree and larger values go to the

right subtree. Equal values use the item id as a tie breaker. Inorder traversal writes `items_sorted_by_value`. Insert is $O(\log n)$ on average and $O(n)$ in the worst case if the tree becomes unbalanced. Traversal and clear are $O(n)$.

VII. ALGORITHMS

A. Greedy Item Selection

The greedy algorithm is implemented in `game/algorithms/greedy.py`. It selects one remaining item to highlight. The decision order is: choose the highest-value item near a guard that is not in a vision zone; otherwise choose the highest-value safe item; if all are dangerous, choose the highest-value remaining item with a warning reason.

Listing 2: Greedy pseudocode.

```
function choose_greedy_item(state):
    items = remaining uncollected items
    if items is empty: return none
    best_near_guard = highest value item near guard
    and safe
    if best_near_guard exists: return best_near_guard
    best_safe = highest value item outside vision
    zones
    if best_safe exists: return best_safe
    return highest value item with warning
```

The algorithm is greedy because it makes one local decision from the current state without exploring future routes. If i is the number of remaining items, g the number of guards, and v the number of vision cells, its complexity is $O(i(g + v))$.

B. Backtracking Route Search

The backtracking algorithm is implemented in `game/algorithms/backtracking.py`. It searches from the current player position to the exit. Invalid cells include out-of-bounds coordinates, walls, cameras, guards, vision zones, and cells already visited in the current route.

Listing 3: Backtracking pseudocode.

```
function backtrack(row, col):
    if cell is invalid: return none
    choose cell: add to route and visited
    if cell is exit: return copy of route
    for each neighbor ordered by distance to exit:
        result = backtrack(neighbor)
        if result exists: return result
    unchoose cell: remove from route and visited
    return none
```

The algorithm follows the choose, explore, unchoose structure. Its worst-case upper bound is exponential, approximately $O(4^n)$, where n is the number of grid cells. In practice, walls and vision zones reduce the search space.

VIII. GAME INTERFACE AND MECHANICS

The UI is implemented in `game/ui/main.py`. It includes start, tutorial, playing, caught, and escaped screens. The playing screen renders the board and sidebar. Rendering order is floor, vision zones, backtracking route, walls, items, selected-item outline, exit, cameras, guards, and player. Fig. 3 shows

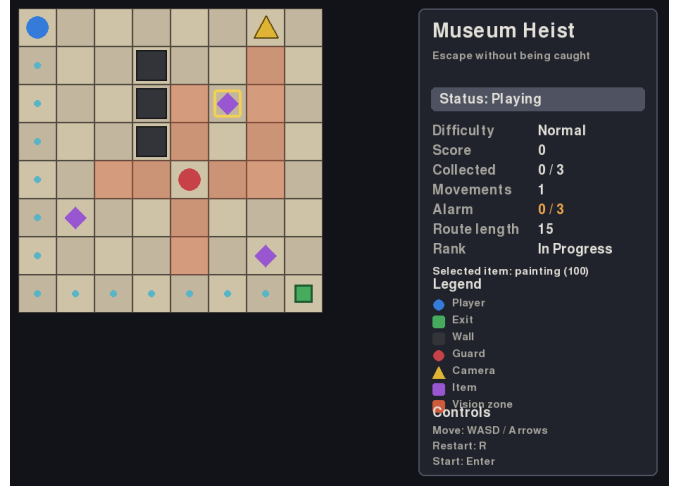


Fig. 3: Gameplay screen with sidebar metrics, route markers, vision zones, objects, player, camera, guard, and exit.

the improved UI with difficulty, score, route length, alarm, and board elements.

The player moves with arrow keys or WASD. The start screen lets the player choose Easy, Normal, or Hard. The tutorial explains the objective and objects. R restarts the current difficulty and M returns to the main menu. The final screen reports score, items collected, movement count, elapsed time, and rank.

IX. TESTING AND VALIDATION

The testing plan includes compilation, Pygame launch, valid movement, invalid movement, item collection, item disappearance, vision-zone capture, alarm capture, movement-limit capture, time-limit capture, escape success, restart, main-menu return, greedy output, backtracking output, and JSON bridge validation. Table III summarizes the main tests.

TABLE III: Main validation tests

ID	Test	Expected result
T01	Compilation	Engine builds without errors.
T03	Movement	Player moves one valid cell.
T06	Item collection	Score increases and item is collected.
T08	Vision zone	Game becomes caught.
T11	Time limit	Game over by time expiration.
T12	Escape	Status becomes escaped.
T15	Greedy	Function returns selected item and reason.
T16	Backtracking	Function returns a safe route or empty list.

The project was also validated through manual gameplay. A successful run requires selecting a difficulty, navigating the map, avoiding vision zones, collecting items, and reaching the exit. Failure cases are intentionally reachable to demonstrate game-over states.

X. DISCUSSION

The project demonstrates how basic algorithms and data structures can be converted into visible mechanics. Greedy selection gives an immediate recommendation but does not guarantee the best final outcome. Backtracking gives route feasibility under constraints but is exponential in the worst case. The linked list and BST are meaningful because they support movement history and value ordering rather than being isolated examples.

The main limitations are the hardcoded maps, manual JSON parsing in C++, the unbalanced BST, and the fact that the backtracking route optimizes feasibility rather than score, alarm, or time. These limitations are acceptable for a Computer Science I project and also provide clear future work.

XI. CONCLUSION

Museum Heist integrates C++, Python, Pygame, JSON communication, greedy selection, backtracking search, a linked list, and a binary search tree into one playable game. The final version includes difficulty levels, item collection, camera and guard vision, alarm, time and movement limits, victory/defeat states, final metrics, tutorial, restart, and main-menu return. The project fulfills the course objective of using algorithms and data structures as functional game mechanics.

REFERENCES

- [1] C. A. Sierra, *Computer Sciences I: Course Project Description*, Universidad Distrital Francisco Jose de Caldas, 2026-I.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [3] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008.
- [4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [5] Pygame Community, *Pygame Documentation*.